

Python: Introduction & Basic Concepts

1. Introduction and Basics

Python is a **high-level, interpreted, general-purpose programming language** created by Guido van Rossum, first released in 1991. It is known for its simple, readable syntax that closely resembles plain English, which makes it a popular choice for beginners as well as professionals.

Key Characteristics:

- **Interpreted:** Code runs line-by-line through an interpreter (no separate compilation step required before execution), unlike compiled languages such as C or C++.
- **Dynamically Typed:** You don't need to declare a variable's type explicitly — Python figures it out at runtime.
- **High-Level:** Manages memory and low-level details (like memory allocation) automatically, letting the programmer focus on logic.
- **Multi-Paradigm:** Supports procedural, object-oriented, and functional programming styles.
- **Extensive Standard Library:** Comes with built-in modules for file handling, math, networking, data structures, and more — often summarized as "batteries included."
- **Cross-Platform:** Runs on Windows, macOS, and Linux without modification.

Common Use Cases: Web development (Django, Flask), data science and machine learning (NumPy, Pandas, TensorFlow), automation/scripting, software testing, and increasingly, backend systems for applications like ERPs.

2. Setting Up Python (Installation & PATH)

To run Python code on a machine, you need the Python interpreter installed and accessible from the command line.

Installation Steps (Windows example):

1. Download the installer from the official source (python.org).
2. Run the installer — **check the box "Add Python to PATH"** during setup (this is the most commonly missed step).
3. Complete installation and verify by opening Command Prompt/Terminal and typing:
4. `python --version`

Why the PATH Variable Matters: The PATH is a system environment variable that tells the operating system where to look for executable programs when you type a command. If Python

isn't added to PATH, typing `python` in the terminal will result in a "command not found" error, even though Python is installed on the disk — because the OS doesn't know where to find it.

Verifying Setup:

- `python --version` → should print the installed version (e.g., Python 3.12.1)
- `pip --version` → confirms the package manager is also correctly set up

3. Data Variables and Their Types

A **variable** is a named location in memory used to store a value that can change during program execution.

```
name = "Sohrab"
age = 21
gpa = 8.7
is_active = True
```

Variable Naming Rules:

- Must start with a letter or underscore (`_`), not a digit
- Can contain letters, digits, and underscores only (no spaces or special characters)
- Case-sensitive (`Age` and `age` are different variables)
- Cannot be a reserved keyword (e.g., `class`, `if`, `for`)

Valid examples: `student_name`, `_temp`, `marks2` **Invalid examples:** `2marks` (starts with digit), `student-name` (hyphen not allowed), `class` (reserved keyword)

Common Built-in Data Types:

Type	Description	Example
<code>int</code>	Whole numbers	<code>age = 21</code>
<code>float</code>	Decimal numbers	<code>gpa = 8.7</code>
<code>str</code>	Text/string	<code>name = "Sohrab"</code>
<code>bool</code>	True/False values	<code>is_active = True</code>
<code>list</code>	Ordered, mutable collection	<code>marks = [80, 90, 75]</code>
<code>tuple</code>	Ordered, immutable collection	<code>point = (2, 3)</code>
<code>dict</code>	Key-value pairs	<code>student = {"name": "A", "age": 21}</code>

4. `id()` and `type()` Functions

- `id(object)` — returns the unique memory address (identity) of an object, as an integer. Every object in Python has a unique `id()` for the duration of its lifetime.
- `type(object)` — returns the data type/class of an object.

```
x = 10
print(type(x))    # <class 'int'>
print(id(x))      # e.g., 140732882313328 (varies by run/machine)
```

These functions are especially useful for debugging and for understanding how Python handles variables internally (e.g., checking whether two variables point to the same object in memory).

5. Coding Standards (PEP 8)

PEP 8 is Python's official style guide, promoting readable and consistent code across projects and teams.

Key Conventions:

1. **Indentation:** Use 4 spaces per indentation level (not tabs).
2. **Naming:** Use `snake_case` for variables and functions (e.g., `student_name`), not `camelCase`.
3. **Line Length:** Keep lines under 79–99 characters where practical.
4. **Comments:** Use `#` for inline comments and docstrings (`""" . . . """`) for functions/classes to explain purpose.
5. **Meaningful Names:** Choose descriptive variable/function names over short, ambiguous ones (`total_marks` instead of `t`).
6. **Whitespace:** Use blank lines to separate logical sections of code; avoid extra spaces inside brackets/parentheses.

Following these standards makes code easier to read, debug, and maintain — especially important in collaborative or long-term projects.